

第二章概论

1.

现有一个长度为正整数 n 的顺序表，请提供伪代码，将该顺序表中的重复元素删除，并给出算法的时间复杂度和空间复杂度

以下为C++实现代码

```
#include <vector>
#include <unordered_set>

std::vector<int> func(const std::vector<int>& s) {
    std::unordered_set<int> visited;
    std::vector<int> res;
    for (int item : s) {
        if (visited.find(item) == visited.end()) { // 如果不在已经找过的里面
            visited.insert(item);
            res.push_back(item);
        }
    }
    return res;
}
```

时间复杂度最坏的情况下时间复杂度是 $O(n)$ ，因为每个元素最多被检查一次，并且集合的插入和查找操作应该是 $O(n)$ ，因此时间复杂度是 $O(n)$

空间复杂度是 $O(n)$ ，因为需要额外集合和vector存储元素

2.

在一个双向链表中，请给出在 p 节点之后插入一个新节点 q 的操作伪代码

```
void insert(Node* p, Node* q) { // 这里使用node来代表双向链表中的节点的数据结构，其包含元素有前向节点和后向节点指针和数据
    q->next = p->next;
    if (p->next != nullptr) {
        p->next->prev = q;
    }
    p->next = q;
    q->prev = p;
}
```

3.

请设计算法，在不修改链表中元素的情况下，判断一个带有头结点head的单向链表是否含有环；如果有环，请找出环的入口点

- 使用快慢指针的方法，首先创建两个指针slow和fast，都指向链表的head。
- 之后进入while循环，在循环内slow每次移动一步，fast每次移动两步`fast.next.next`，如果两者相遇，就说明链表中存在环，如果不相遇，也就是fast或fast.next为空，则没有环，将这个作为循环条件。
- 找入口：检测到环存在，将slow指针重新指向head，进入循环，条件是slow不等于fast，本循环中slow和fast每次都移动一步，它们再次相遇的时候，相遇点就是环的入口